Longest Previous Non-Overlapping Factors Computation

Supaporn Chairungsee
Department of Information Technology
School of Informatics, Walailak University
Nakhonsithammarat, Thailand
supaporn.ch@wu.ac.th

Tida Butrak, Surangkanang Chareonrak, Thana Charuphanthuset
Department of Information Technology
Faculty of Science and Technology
Suratthani Rajabhat University
Suratthani, Thailand
{tida_saw, surang_sru, thanac}@hotmail.com

***Abstract*—We study the problem of finding the Longest Previous non-overlapping Factor (LPnF) occurring at each position of a string. The notion of LPnF table is a variant of the Longest Previous Factor (LPF) table and is an essential element for the design of efficient algorithms on strings. The LPnF table is related to Ziv-Lempel factorization which is used for text compression. In this paper, we describe an algorithm for computing the LPnF table of a string from its Suffix Automaton. The algorithm runs in linear time on a fixed size alphabet and applications of this algorithm are for text compression.**

Keywords—Suffix Automaton; longest previous non-overlapping factor; factorisation; text compression; data compression

I. INTRODUCTION

The problem is to compute the Longest Previous non-overlapping Factor (LPnF) table for a given string y in an efficient way. The Longest Previous non-overlapping Factor table (LPnF) stores for each position of a string the maximal length of factors occurring both there and in the preceding part of the string. The LPnF table of the string y = abbaabbbaaab is shown in the following:

Position i	0	1	2	3	4	5	6	7	8	9	10	11
$y[i]$	a	b	b	a	a	b	b	b	a	a	a	b
LPnF[i]	0	0	1	1	3	2	4	3	2	3	2	1

The concept of LPnF table is similar to the Longest Previous Factor (LPF) table [9, 10, 11]. This table extends the Ziv-Lempel factorization of a text [15, 16, 18] which is useful for conservative text compression [3]. The f-factorization plays an important role in String Algorithms especially for on-line string processing. An application of this approach resides in algorithms to compute repetitions in strings [7, 14]. Let us consider the algorithm in [14], this algorithm presents all maximal repetitions that appear in a string in $O(n \log |A|)$ time. Furthermore, the concept of LPnF table is closed to the Longest Previous reverse Factor (LPrF) table [1, 5]. The LPrF table generalizes a factorization of strings to extract palindromes in molecular sequences. These palindromes play an important role in RNA secondary structure prediction and text compression.

The motivation to consider the LPnF table is text compression. The reason is it may be used to improve the Ziv-Lempel factorization which is the basic of several well-known compression applications [17]. The notion of Longest Previous non-overlapping Factors is a variant of Longest Previous Factor, where the previous occurrences of a factor could not overlap the current occurrence. A linear-time solution for computation based on the Suffix Array of the text is described in [10].

The problem of LPnF table computation is related to the factorization since it is an extension of a factorization of strings that is used to design algorithms on strings [8]. The factorization is close to the Ziv–Lempel factorization [18] that is used for text compression [3]. The LPnF table can be improved the string algorithms and text compression methods whereas the cost of computation is lower than the cost of the f-factorization. The solution presented in [11] is based on the deep analysis of the difficulties of LPnF computation over LPF computation. Then it does not require any other sophisticated data structures like indexes for efficiently solving the range minimum query problem.

In this paper, we present algorithms for computing efficiently the LPnF table of a string from its Suffix Automaton. The algorithm runs in linear time, that is, $O(n)$ time for a string of length n, provided the alphabet is of fixed size. On a general alphabet the running time becomes $O(n \log a)$ where a is the number of different letters occurring in the input string. The factorizations of strings used for text compression and for designing string algorithms may be further optimized. Our algorithms apply the notion of suffix links of the data structures that are essential for their linear-time construction.

II. BASIC DEFINITIONS

In this section, we firstly recall the notion of string. Next, we recall the notion of factor. Then, we recall the notion of Suffix Automaton which is a useful data structure for efficiently solving the string matching problem and other string problems. After that we describe the formally definition of the LPnF table. Finally, we provide the Knuth-Morris-Pratt algorithm.

1529-4188/15 $31.00 © 2015 IEEE
DOI 10.1109/DEXA.2015.21

A. String

An alphabet A denotes a finite nonempty set and the elements of an alphabet are called characters, letters or symbols [4, 8]. A string s over an alphabet A is a (finite) concatenation of sequence of elements of A. The length of a string s is the number of symbols in s, it is denoted by $|s|$. The empty string (e) denotes the string which has the length 0. For example, string s = abaab is a string of length five over the alphabet A = {a, b}.

B. Factor

A string x is a factor of a string y if there exist two strings u and v such that $y = uxv$ [8]. For example, string x = aba is a factor of y = aababaabab.

C. Suffix Automaton

Let $S(w)$ denotes the Suffix Automaton of a given string w [8]. It is the minimal automaton which corresponds to the set of suffixes of w. For instance, the Suffix Automaton of the string w = AGGTTAC is presented in Figure 1.

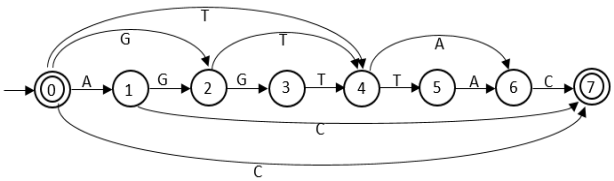

Fig. 1. Suffix Automaton of the string AGGTTAC.

The structure includes the table L and the failure link F. These structures are defined as follows. Let L is the table to store the maximum length of labels of paths from the initial state to the current state. $L[q]$ denotes the maximum length of labels of paths from the initial state to state q. For example, the value of $L[5]$ of Figure 1 is 5. The next table gives the attributes L of the states of the automaton displayed in Figure 1:

State q	0	1	2	3	4	5	6	7
$L[q]$	0	1	2	3	4	5	6	7

Let F is called the failure link of the states of the automaton and $F(q)$ denotes the suffix target of state q as the equivalence class of $s(u)$. The value of $s(u)$ is independent of the string u chosen in the class of factors of y equivalent. For example, the suffix target of state 6, $F[6]$, of Figure 1 is state 1. The table below gives the attributes F of the states of the automaton displayed in Figure 1:

State q	0	1	2	3	4	5	6	7
$F[q]$	0	0	0	2	0	4	1	0

D. Longest Previous non-overlapping Factor

We study a string y of positive length n on the alphabet A where $y = y[0..n - 1]$ [12]. The problem is to compute the Longest Previous non-overlapping Factor table and this table is defined, for $0 = i < n$, by

$$\text{LPnF}[i] = \max \{k \mid y[i..i + k - 1] \text{ appears in } y[0..i - 1]\}.$$

For instance, the LPnF table for the string y = aababaab is presented as follows:

Position i	0	1	2	3	4	5	6	7
$y[i]$	a	a	b	a	b	a	a	b
LPnF[i]	0	1	0	2	2	3	2	1

According to the above table, if we consider the value of LPnF table at position 3, we found that the factor ab which started at position 1 is the longest factor. The length of this factor is 2 as a result the value of LPnF[7] is equal to 2.

The problem of the LPnF table computation is closed to the computation of the Longest Previous Factor (LPF) table, the table to store the maximal length of the previous factor occurring at each position of the text [10, 17], and the Longest Previous reverse Factor (LPrF) table, the table to store the maximal length of factors occurring both there and in the preceding part of the string [1, 5].

For example, the below table displays the LPnF, LPF and LPrF of the string y = ababaaabbaab. Let us consider position 2, the LPnF[2] is equal to 2 as ab is the longest prefix that appears at a smaller position. The LPF[2] is equal to 3 since aba is the longest prefix that appears at a previous position. While the LPrF[2] is equal to 1 because a is the longest prefix that appears reverse at a smaller position.

Position i	0	1	2	3	4	5	6	7	8	9	10	11
$y[i]$	a	b	a	b	a	a	a	b	b	a	a	b
LPnF[i]	0	0	2	2	1	1	2	1	3	3	2	1
LPF[i]	0	0	3	2	1	2	2	1	3	3	2	1
LPrF[i]	0	0	1	2	1	1	2	1	3	3	2	1

E. Knuth-Morris-Pratt Algorithm

The pattern is a nonempty language that does not contain the empty string [8]. The problem of pattern matching (or exact string matching or text searching in other contexts) is the problem to find the occurance of the pattern as a substring in the given string [13, 18]. Knuth-Morris-Pratt (KMP) algorithm is a linear-time algorithm for the pattern matching problem and the matching time of this algorithm is $O(n)$ because it does not compare the elements of the string that have previously been matched with the element of the pattern [6, 13].

III. METHOD

In this section, we present two algorithms to compute the LPnF table. Firstly, we demonstrate how to compute the LPnF table using the KMP algorithm where we call this algorithm as the naive algorithm LPnF-KMP. Next, we present a linear-time approach to compute the LPnF table of the input string y. Our algorithm applies the useful data structure, Suffix Automaton.

A. Using KMP to compute LPnF

In this subsection, we present the computation of the LPnF table by the naive algorithm LPnF-KMP. For two strings u and v the function KMP(u, v) returns the length of the longest prefix of the string u that is a factor of the string v. When we compute LPnF[i], we use KMP algorithm to find the longest prefix of $y[i..n-1]$ that is a factor of the string $y[0..i-1]$. The algorithm LPnF-KMP is presented in the following.

```
LPnF-KMP (y, n)
1  for i ← 0 to n − 1 do
2      LPnF[i ] ← KMP(y[i..n − 1], y[0.. i − 1])
3  return LPnF
```

The running time of KMP($y[i..n-1]$, $y[0..i-1]$) in line 2 of the LPnF-KMP algorithm is $O(n)$. It can be reduced to $O(\max\{i, n - i\})$. Therefore, we compute LPnF[i] of string y for n times, the overall running time for this algorithm is quadratic ($O(n^2)$).

B. Using Suffix Automaton to compute LPnF

In this subsection, we present a linear-time approach for the LPnF table computation of the input string y. Our algorithm uses the Suffix Automaton, $S(y)$, of the input string. The structure includes the failure link (F) and the table (L). Figure 2 presents the Suffix Automaton of the string babaababaa and this figure is used to compute the LPnF table of the given string babaababaa.

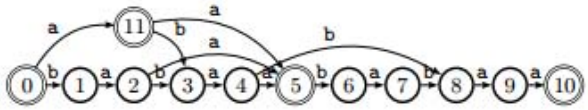

Fig. 2. Suffix Automaton of the string babaababaa.

The table below presents the values of F and L for each states of the Suffix Automaton displayed in Figure 2.

State q	0	1	2	3	4	5	6	7	8	9	10	11
$F[q]$	0	0	11	1	2	11	3	4	3	4	5	0
$L[q]$	0	1	2	3	4	5	6	7	8	9	10	1

The code of the algorithm in the following applies the Suffix Automaton $S(y)$ of the string y. The algorithm works as follows.

At a given step i is a position on y, l is the length of the current match, q is the current state of the automaton, *initial* its state and δ denotes its transition function. The invariant is $\delta(initial, y[i - \ell .. i - 1]) = q$. The invariant shows in the loop of the algorithm LPnF-AUTOMATON. The condition to extend the match by the letter $a = y[i]$ is $\delta(q, a)$ is defined and $y[i - \ell .. i - 1]a$ appears in y at a position before at least as large as $n - i + \ell$. The test becomes $i - \ell \geq L[\delta(q, a)] - l + 1$ where the first member is the length of $y[0..i - \ell - 1]$ and the second member is the minimal length of suffixes of y starting with the next match. If the result of the test is negative, the failure link F is used to shorten the match whose length is given by L. All the suffixes of the match of length smaller than $L[F[q]]$, that correspond to the same state q, is able to change the result of the condition in line 5. Therefore, a batch of LPnF values are computed in lines 10-13.

In the following code we assume that F [*initial*] = *initial*. The value of F [*initial*] is left undefined for Suffix Automata but the assumption simplifies the presentation of the algorithm.

```
LPnF-AUTOMATON (y, n)
1  (q, l) ← (initial, 0)
2  i ← 0
3  repeat
4      a ← y[i]
5      while (i < n) and (δ (q, a) ≠ NULL) and (i - l) ≥
       L(δ (q, a) – l + 1) do
6          (q, l) ← (δ (q, a), l + 1)
7              i ← i + 1
8              a ← y[i]
9      end
10     repeat
11         LPnF[i - l] ← l
12         l ← max {0, l - 1}
13     until l = L[F[q]];
14     if q ≠ initial then
15       q ← F[q]
16       else i ← i + 1
17     end
18 until (i = n) and (l = 0);
19 return LPnF
```

Theorem 1. The algorithm LPnF-AUTOMATON *can compute the* LPnF *table of a string of length n in time O(n) on a fixed size alphabet.*

Proof. We analyze the running time of the algorithm. The first step consists of the construction of the Suffix Automaton including table F and L of its states. All the preprocessing can be done in linear time [8].

The running time of the rest step relies on the number of the tests both in line 5 and line 13. The former either leads increment i or to execute the next instruction. The latter yields an increment of the expression $i - l$. The values of these two expressions never decrease therefore only n of them are executed. If the Suffix Automaton is constructed in linear space, the overall algorithm runs in $O(n \log a)$ time.

IV. Experimental Results

In this section, we present the experimental results for the LPnF table computation using the Suffix Automaton. The data set consists of four RNA sequences that are A.Baumannii, A.Chrysogenum, B. Cereus and B.Longum. Let us consider the memory size for the LPnF table computation. We want to see whether the memory size is a linear function with respect to the length of the string. Experimental results represent that the memory size for the computation of the LPnF table, using the Suffix Automaton is a linear function with the string length. Figure 3 displays the memory size of the LPnF table computation.

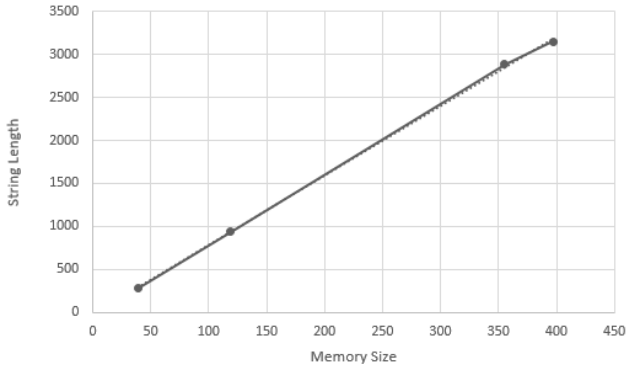

Fig. 3. Memory Size of Experimental Results.

V. Conclusion

We present the notion of the LPnF table of a string which is the table stores the maximal length of factors at each index *i* that start at position *i* on the string *y* and appear at the position before. We also present an algorithm to compute the LPnF table using the Suffix Automaton. Our algorithm runs in linear time on a fixed size alphabet and in $O(n \log a)$ time otherwise in a linear memory space. An interesting open question about this problem is there exists a straight computation of the LPnF table using other data structures and the running time of the computation is in linear time.

References

[1] G. Badkobeh, S. Chairungsee, and M. Crochemore. Hunting redundancies in strings. In G. Mauri and A. Leporati, editors, Developments in Language Theory (DLT 2011), volume 6795 of Lecture Notes in Computer Science, pages 1–14. Springer, 2011

[2] T. C. Bell, J. G. Clearly, and I. H. Witten. Text Compression. Prentice Hall Inc., New Jersey, 1990.

[3] J. Berstel, A. Savelli, and Crochemore factorization of Sturmian and other infinite words, in: R. Kralovic, P. Urzyczyn (Eds.), MFCS 2006, in: LNCS, vol. 4162, Springer, 2006, pp. 157–166.

[4] H. J. B¨ockenhauer and D. Bongartz. Algorithmic Aspects of Bioinformatics. Springer, Berlin, 2007.

[5] S. Chairungsee and M. Crochemore. Efficient computing of longest previous reverse factors. In Seventh International Conference on Computer Science and Information Technologies (CSIT 2009), pages 27–30, 2009.

[6] [17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. Introduction to Algorithms. The MIT Press, Massachusetts, 2009.

[7] M. Crochemore, Transducers and repetitions, Theoretical Computer Science 45 (1) (1986) 63–86.

[8] M. Crochemore, C. Hancart, and T. Lecroq. Algorithms on Strings. Cambridge University Press, Cambridge, UK, 2007.

[9] M. Crochemore and L. Ilie, Computing longest previous factor in linear time and applications, Inf. Process. Lett. 106 (2) (2008) 75–80.

[10] M. Crochemore, L. Ilie, C. Iliopoulos, M. Kubica, W. Rytter, and T. Wale´n, LPF computation revisited, in: D. Fronçek, J. Kratochvíl, M. Miller (Eds.), IWOCA 2009, in: LNCS, vol. 5874, Springer, 2009, pp. 158–169.

[11] M. Crochemore, C. Iliopoulos, M. Kubica, W. Rytter, and T. Wale´n, Efficient algorithms for two extensions of the LPF table: the power of Suffix Arrays, in: J. van Leeuwen, A. Muscholl, D. Peleg, J. Pokorný, B. Rumpe (Eds.), SOFSEM 2010, in: LNCS, vol. 5901, Springer, Berlin, 2010, pp. 296–307.

[12] M. Crochemore and G. Tischler, Computing Longest Previous nonoverlapping Factors, Inf. Process. Lett. 111 (2011) 291–295.

[13] D. E. Knuth, J. H. Morris, and V. R. Pratt. Fast pattern matching in strings. Society for Industrial and Applied Mathematics Journal on Computing, 6:323–350, 1977.

[14] R. Kolpakov and G. Kucherov, Finding maximal repetitions in a word in linear time, in: Foundations of Computer Science, 1999, pp. 596–604.

[15] I. M. Pu. Fundamental Data Compression. A Butterworth-Heinemann, 2006.

[16] J. A. Storer. Data Compression: Methods and Theory. Computer Science Press,1988

[17] I. H. Witten, A. Moffat, and T. C. Bell. Managing Gigabytes. Van Nostrand Reinhold, New York, 1994.

[18] J. Ziv and A. Lempel. A universal algorithm for sequential data compression. IEEE Transactions on Information Theory, pages 337–343, 1977.